

Career Compass: A Skill-Based Job Recommendation Platform

Team Workforce Insights — SIADS 699 Capstone

Cory Vinlove, Tameem Syed

Project Statement

Job seekers routinely face a practical problem that has nothing to do with whether they are qualified for a role: job postings describe the same skills and qualifications using wildly inconsistent terminology. One posting asks for "data analysis," another for "analytics," a third simply lists "Excel, SQL, Tableau" with no summary label at all. This inconsistency makes it hard for a job seeker to efficiently compare postings, to recognize when a role is actually a strong fit, or to identify which specific skills stand between them and a role they want.

Career Compass addresses this problem directly. Given a set of skills a user already has, the platform recommends the job postings most relevant to that skill set, and separately identifies which skills commonly appear in similar postings that the user does not yet have — a skill gap. The underlying question we set out to answer was not primarily "can we build a recommender," but "can we get from unstructured, inconsistent job-posting text to something a job seeker can actually act on, using only what's in the posting itself." That constraint — no resume database, no historical application data, no user click history — shaped every methodological choice described below.

We built the platform on a public dataset of 123,849 LinkedIn job postings, scraped in early 2024, covering titles, full free-text descriptions, company metadata, and (sparse) salary information. This dataset does not, on its own, tell you what skills a posting requires — that had to be extracted.

Methodology

Data and its real limitations

Before any modeling, we ran two exploratory passes over the data (documented in 02_eda_postings.ipynb and 03_eda_companies_and_jobs.ipynb). The most consequential finding was how little of the data is directly usable in structured form. Several fields we initially expected to lean on are mostly empty: `skills_desc` (98.0% missing), the salary fields (71–95% missing depending on the specific column), and `remote_allowed` (87.7% missing) (Figure 1). The dataset's own structured skill field, `job_skills.csv`, is populated for 98.6% of postings, but only maps each posting to one of 35 broad functional categories such as "Engineering" or "Sales" — not to actual skills like Python or AutoCAD.

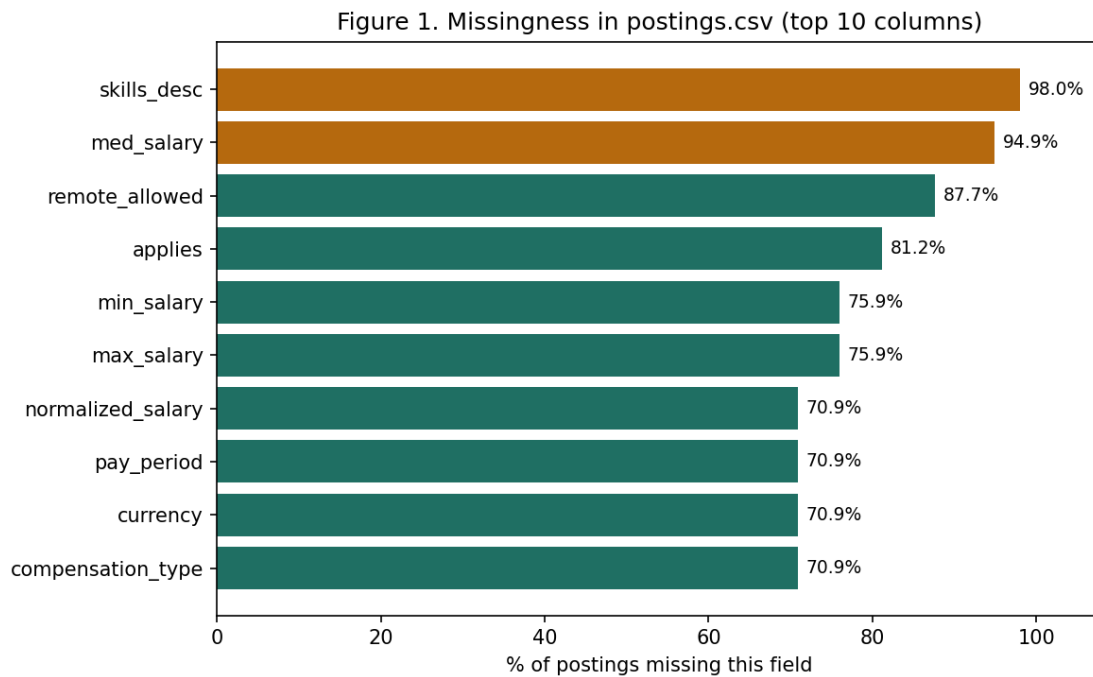


Figure 1. Missingness in postings.csv (top 10 columns). Red bars indicate fields missing 90%+ of values.

This finding directly determined our methodology: any skill-based feature had to be extracted from the free-text description field, because no structured alternative in the dataset was fine-grained enough to use.

Before extraction, we resolved a set of data-quality issues surfaced during EDA, each with evidence rather than assumption. `remote_allowed` never contains an explicit "false" value in the raw data — only 1.0 or missing — so we treated missing as "not marked remote" rather than an ambiguous gap. Salary data contains a small number of clear errors (a maximum `normalized_salary` of \$535.6 million against a median of \$81,500); rather than discard those postings outright, we flagged 469 of 36,073 USD salary rows (1.3%) as unreliable and kept the rest of each posting's data intact, since salary is only one of many fields relevant to a recommendation. We also found that `companies.csv` uses the literal strings "0" and "-" in place of true nulls in several location fields, affecting up to 3,972 rows depending on the column — a pattern standard missingness checks would not catch on their own.

Skill extraction

We extracted skills using a curated gazetteer of 133 terms across ten categories (programming languages, data/ML tools, cloud platforms, business and soft skills, healthcare, finance, and others), matched against posting descriptions with `flashtext`, a trie-based multi-keyword search library. This is a deliberate, explainable choice for a first version: every match is traceable to an exact term, it requires no labeled training data, and its precision can be checked by hand — properties that matter when we have no ground truth to validate against. The tradeoff is coverage: the approach only finds skills we thought to include, and cannot recognize paraphrased mentions the way a named-entity recognition (NER) or large-language-model-based approach could. Recent NLP research on this exact task, including Zhang et al.'s SkillSpan benchmark and subsequent LLM-based approaches (Nguyen et al., 2024), treats skill extraction as a sequence-labeling problem precisely to move beyond fixed vocabularies — we view that as the natural next iteration once a labeled evaluation set exists, rather than a change we could justify making blind.

Our first implementation of the gazetteer matcher used a single large regular expression combining all skill terms. At the full dataset's scale (123,849 documents), this proved computationally impractical — early benchmarking projected close to an hour of runtime, because backtracking regex evaluation scales with the number of alternatives at every character position. We replaced it with `flashtext`, whose Aho-Corasick-style trie structure finds all matches in a single pass regardless of vocabulary size; the full run completed in 95 seconds. We note this not as a footnote but because it is a real methodological decision: choosing the right algorithm, not just the right feature,

mattered at this data volume.

The extractor found at least one skill in 88.6% of postings, averaging 3.09 skills per posting (Figure 2). The most frequently mentioned skills are broad, cross-industry soft skills — Leadership, Communication, Sales — which is consistent with the dataset spanning all industries rather than only technology roles.

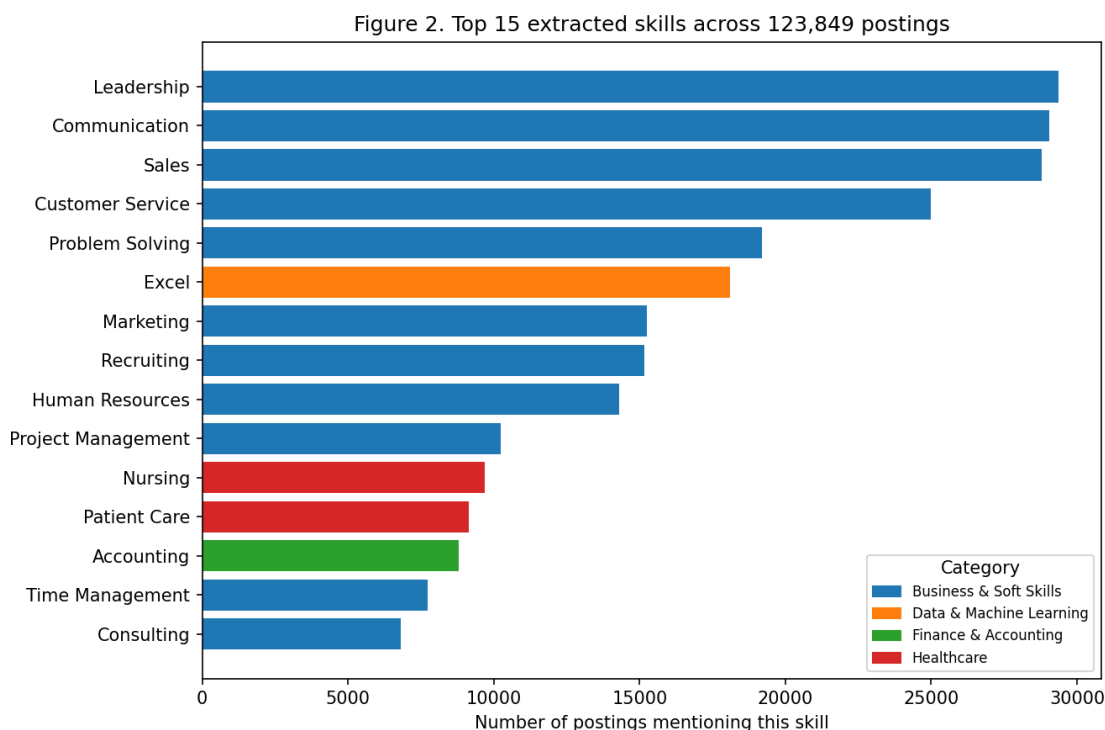


Figure 2. Top 15 extracted skills across 123,849 postings, colored by category.

Recommendation methodology

We represent each posting as a binary vector over the 133 gazetteer skills, and a user profile as a vector of the same shape. Because common skills like Leadership appear in tens of thousands of postings while specific skills like AutoCAD or HIPAA appear in only a few hundred, an unweighted overlap score would let two postings match mainly on their shared soft-skill mentions, obscuring more meaningful, specific matches. We addressed this with inverse document frequency (IDF) weighting — the same principle behind TF-IDF in text retrieval, applied here to a skill vector rather than a word vector — so that rarer, more specific skills contribute proportionally more to the similarity score. Recommendations are ranked by cosine similarity between the weighted user vector and each weighted posting vector.

This is a content-based filtering approach, not collaborative filtering. That choice was constrained by the data available, not preference: collaborative filtering requires user interaction history (clicks, applications, ratings) that does not exist in this dataset. A recent systematic literature review of job recommender systems (Springer, *Journal of Big Data*, 2025) confirms this is the standard tradeoff in the field — content-based methods are the default when interaction data is unavailable, with hybrid approaches becoming viable only once some interaction signal exists.

Evaluation Strategy

No ground-truth "correct recommendation" labels exist for this dataset, which constrains what a rigorous

evaluation can claim. We used two complementary checks. First, a qualitative check: we constructed three realistic user profiles spanning distinct domains (a Data Analyst profile with Python/SQL/Excel/Data Analysis; a Nurse profile with Nursing/Patient Care/HIPAA; a Marketing profile with Marketing/Digital Marketing/SEO) and manually reviewed whether the top-10 recommended postings for each were plausibly relevant. In all three cases, recommended titles matched the intended domain (e.g., "Data Analyst," "Senior Data Analyst," and "Business Analyst" for the Data Analyst profile).

Second, a quantitative check: for each profile, we compared the average similarity score of the top-10 recommended postings against the average similarity of 10 randomly sampled postings. If the recommender were doing nothing meaningful, these two numbers would be close. They are not: the top-10 recommendations scored 16 to 37 times higher in average similarity than the random baseline across the three profiles (Figure 4). This does not prove the recommendations are optimal, but it does demonstrate the ranking mechanism is doing substantive, non-arbitrary work — a necessary, if not sufficient, condition for a useful recommender.

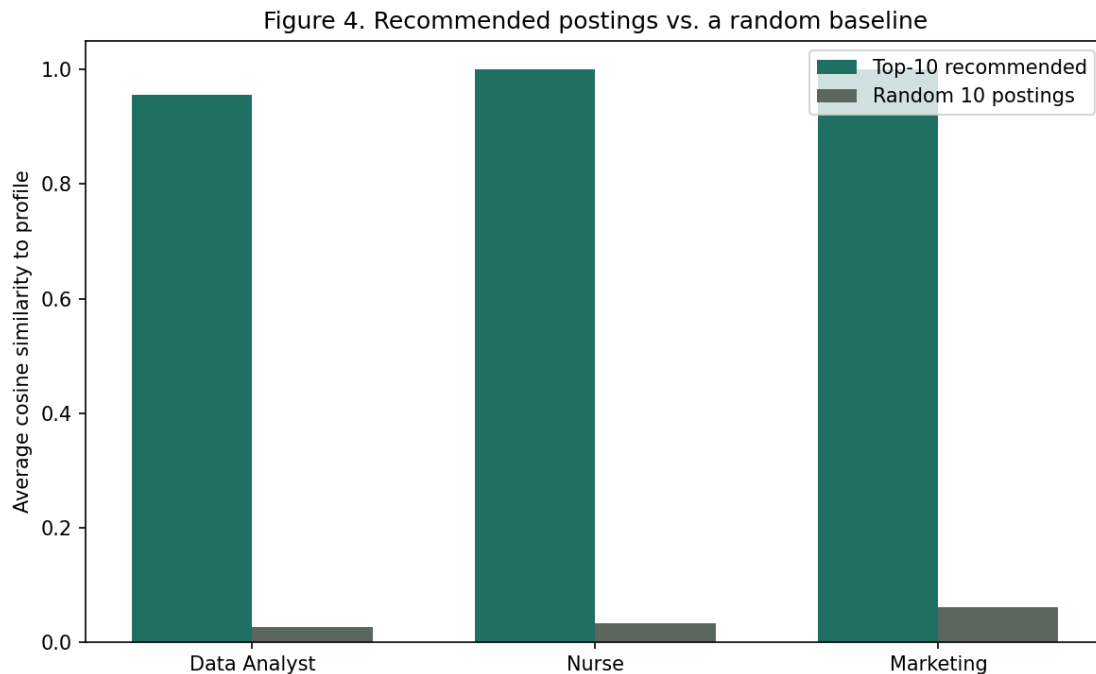


Figure 4. Average recommendation similarity for three test profiles vs. a random-postings baseline.

Results and Technical Depth

The complete pipeline — exploratory analysis, data cleaning with documented rationale, skill extraction, and a working recommendation engine with a functioning dashboard — runs end to end on the full 123,849-posting dataset. Technically, this project draws on natural language processing (gazetteer-based information extraction, with the tradeoffs of dictionary methods versus sequence-labeling NER made explicit), applied statistics (IDF weighting, cosine similarity, percentile-based outlier detection grounded in the actual salary distribution rather than an arbitrary threshold), and software engineering under real performance constraints (diagnosing and resolving an algorithmic bottleneck rather than simply accepting a slow first implementation). The dashboard (app.py, built with Streamlit) exposes the recommendation engine as an interactive tool: a user selects from the 133 recognized skills and receives ranked postings and a skill-gap list in real time.

Translation

What does a similarity score of, say, 0.85 actually mean to a job seeker? In practical terms: of the skills our system recognizes in that user's profile, weighted by how distinctive each one is, this posting shares a large share of the most distinctive ones. The skill-gap output translates this further — for a Data Analyst profile, the system surfaces skills like "Tableau" or "Statistics" that appear frequently among the postings it already ranked highly, meaning the platform is not just saying "these jobs are close," but "these specific additional skills would make you a stronger match for exactly these kinds of roles." That is a materially more actionable output than a generic list of trending skills, because it is conditioned on the user's actual existing profile and on real postings currently in the market, not an abstract industry survey.

Broader Impacts

The most direct beneficiaries of this platform are individual job seekers, particularly those navigating a job market outside their immediate professional network, where informal knowledge about "what skills actually matter for this kind of role" is harder to access. Secondary beneficiaries include career counselors and educators who could use skill-gap patterns in aggregate to inform curriculum or advising decisions.

The central ethical concern is one this project shares with every algorithmic hiring and recommendation tool: job postings reflect the hiring practices, compensation structures, and — potentially — the biases of the employers who wrote them. A recommendation system trained or matched on this data can reproduce those patterns rather than correct them. This is not a hypothetical risk; documented cases such as Amazon's discontinued internal hiring algorithm, which learned to downrank resumes associated with women because historical hiring data skewed male, illustrate how directly historical data bias can translate into biased algorithmic output (as discussed in Raghavan et al.'s and related work on fairness in recruitment-domain recommender systems). Our system differs from that case in an important way — it recommends postings to job seekers rather than ranking candidates for employers, so it cannot directly deny anyone an opportunity the way a resume-screening tool can. But it can still under- or over-represent certain roles or skills for certain users if the underlying posting data itself is skewed, and we have not audited for that in this version.

We mitigate this in two concrete ways, both already reflected in the current build. First, transparency: every recommendation is explainable in terms of specific matched skills, not a black-box score, so a user (or an auditor) can see exactly why a posting was suggested. Second, framing: the platform presents skill gaps as informational context, not a determination that a user is unqualified — a deliberate choice to avoid overstating what a similarity score can tell someone about their actual employability. We did not collect or store any user data beyond the skills entered in a single session, which avoids a second-order privacy risk common to tools in this space.

Limitations and Future Work

The gazetteer's 133 skills, while broad, is not exhaustive, and the matching approach cannot recognize a skill described in unfamiliar phrasing. Expanding this using an established taxonomy (O*NET or ESCO) or a trained NER model, evaluated against the current gazetteer output as a baseline, is the clearest next step. The recommendation engine has no mechanism for result diversity and could plausibly surface many near-duplicate postings from a single high-volume employer. Most importantly, we have not evaluated this system with real users — every check in this report is either a manual sanity check by the team or a comparison against a random baseline, and neither substitutes for observing whether an actual job seeker finds the recommendations useful.

Statement of Work

Tameem Syed led implementation of the data pipeline: exploratory data analysis across both the core postings table and the supporting company/job/mapping tables, the data-cleaning notebook and its documented decisions, the skill-extraction pipeline (including diagnosing and resolving its initial performance bottleneck), the recommendation engine and its evaluation, the Streamlit dashboard, and repository setup and maintenance. This work was completed with substantial assistance from Claude (Anthropic); see the repository README's Code

Attribution section for detail on what that assistance covered.

Cory Vinlove contributed to initial data source evaluation during the proposal phase and is contributing supplementary documentation on project construction and communication, intended to complement this report.

This statement of work reflects contributions as of this draft and will be updated before final submission.

References

- Nguyen, K., et al. (2024). Rethinking Skill Extraction in the Job Market Domain using Large Language Models. *Proceedings of the 1st Workshop on NLP for Human Resources (NLP4HR)*. ACL Anthology. <https://aclanthology.org/2024.nlp4hr-1.3/>
- Zhang, M., et al. (2022). Skill Extraction from Job Postings using Weak Supervision. *NAACL 2022*. ACL Anthology. <https://aclanthology.org/2022.naacl-main.366.pdf>
- (2024). Deep Learning-based Computational Job Market Analysis: A Survey on Skill Extraction and Classification from Job Postings. *arXiv:2402.05617*. <https://arxiv.org/abs/2402.05617>
- Job recommender systems: a systematic literature review, applications, open issues, and challenges. (2025). *Journal of Big Data*, 12. Springer Nature. <https://link.springer.com/article/10.1186/s40537-025-01173-y>
- Fairness of recommender systems in the recruitment domain: an analysis from technical and legal perspectives. *PMC*. <https://www.ncbi.nlm.nih.gov/pmc/articles/PMC10587596/>
- Ethics and discrimination in artificial intelligence-enabled recruitment practices. (2023). *Humanities and Social Sciences Communications*. Nature. <https://www.nature.com/articles/s41599-023-02079-x>

Appendix: Use of AI in This Project

This project made substantial use of Claude (Anthropic) as a coding assistant, so we are documenting that use directly rather than leaving it implicit in the Code Attribution note in our README.

How AI fit into the workflow. Development was an iterative loop, not a single generation step (Figure 5). For each stage of the pipeline, the team set the scope and constraints, Claude drafted and executed the corresponding code, and the team reviewed the actual output — executed notebook results, error logs, rendered figures — before the next stage began. Several points in this project involved the team (via prompts) explicitly withholding approval until evidence was shown: for example, cleaning decisions were only made after missingness and outlier evidence was presented, not before.

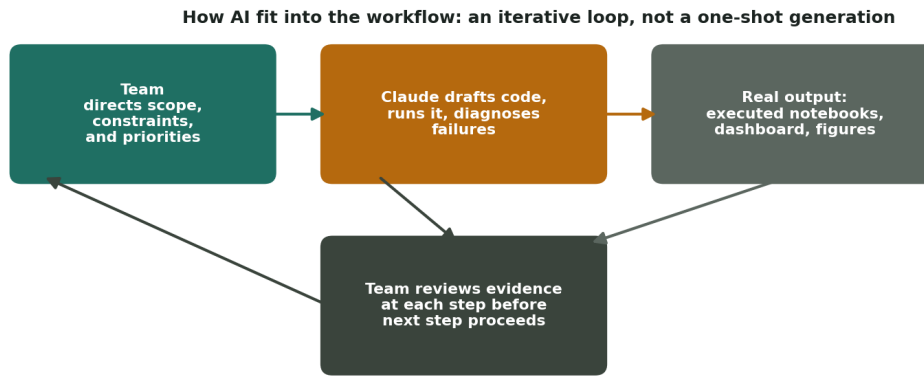


Figure 5. How AI fit into the development workflow - an iterative loop with team review at each stage.

Model and parameters. We used Claude Sonnet 5 (model ID `claude-sonnet-5`) via Claude Code, Anthropic's CLI-based coding agent. The primary tunable parameter we set was reasoning effort, configured at a moderate level (40 on Anthropic's internal 0-100 scale) rather than the maximum available, reflecting a deliberate tradeoff toward faster iteration over exhaustive reasoning on most tasks in this project. Temperature and other decoding parameters were left at the harness's defaults.

How prompts were composed. Prompts generally stated the current project state, an explicit task, and explicit constraints on what should or should not happen automatically, rather than open-ended requests. An actual example, verbatim, from early in the project:

"I am starting a data science capstone project using this LinkedIn job postings dataset. My main table is `postings.csv`, with related tables in the `companies`, `jobs`, and `mappings` folders. Help me create an initial EDA notebook. Start only with `postings.csv`. I want to inspect shape, data types, duplicates, missingness, and important categorical and numerical distributions. Do not build any machine learning models yet. Explain each step and do not make major cleaning decisions without showing me the evidence first."

Note the explicit scope limit (`postings.csv` only), explicit prohibition (no modeling yet), and explicit process requirement (show evidence before deciding) — this pattern of constraint-setting was used throughout the project rather than only at the start.

How outputs were evaluated. Every notebook was actually executed end-to-end (not merely written) and checked for zero runtime errors before being treated as complete; numeric outputs were sanity-checked against the underlying data rather than accepted on inspection alone; and results that seemed wrong or impractical were treated as failures requiring diagnosis, not shipped anyway. A concrete example: the first implementation of the skill-extraction pipeline (a single combined regular expression over roughly 90 skill terms) was benchmarked on a subset of the data before being run on the full 123,849-posting dataset. That benchmark projected a runtime of close to an hour for the full dataset. This was treated as a failed evaluation, not an acceptable slow result — it prompted a diagnosis of the underlying algorithmic cause (regex alternation scales with the number of terms at every character position) and a switch to `flashtext`, a trie-based multi-keyword matching library, which was re-benchmarked (95 seconds on the full dataset) and re-validated against a manual sample of postings before being accepted into the pipeline. A second, lighter-weight example: draft versions of this report's PDF were rendered to images and visually inspected page by page before being finalized, which caught and fixed a text-wrapping defect that plain code review would not have surfaced.

A full prompt log was not maintained as a separate file in this repository.